

Lecture 2

Software quality factors

Outline

- Software quality factor
- McCall's Factor Model
- Alternative models for quality factors

References

- Daniel Galin, Software Quality Assurance: From theory to implementation, chapter 3.1–3.5

Software quality factor

What is Software quality factor?

A set of attributes grouped into **content groups** that define the operational, revision, and transition characteristics of software

The Scope of Requirements:

- **Functional:** What the system *does*. Captured via Use Cases.
- **Quality Factors:** How the system *behaves*. Captured in the Software Requirements Specification (SRS)

What is Software quality factor?

Different SQA models suggest 11-15 factors categorized

Quality is not “one size fits all”. The importance of specific factors varies based on the unique goals of each project

For example: Medical software demands extreme **Reliability**, whereas a mobile game emphasizes **Usability**

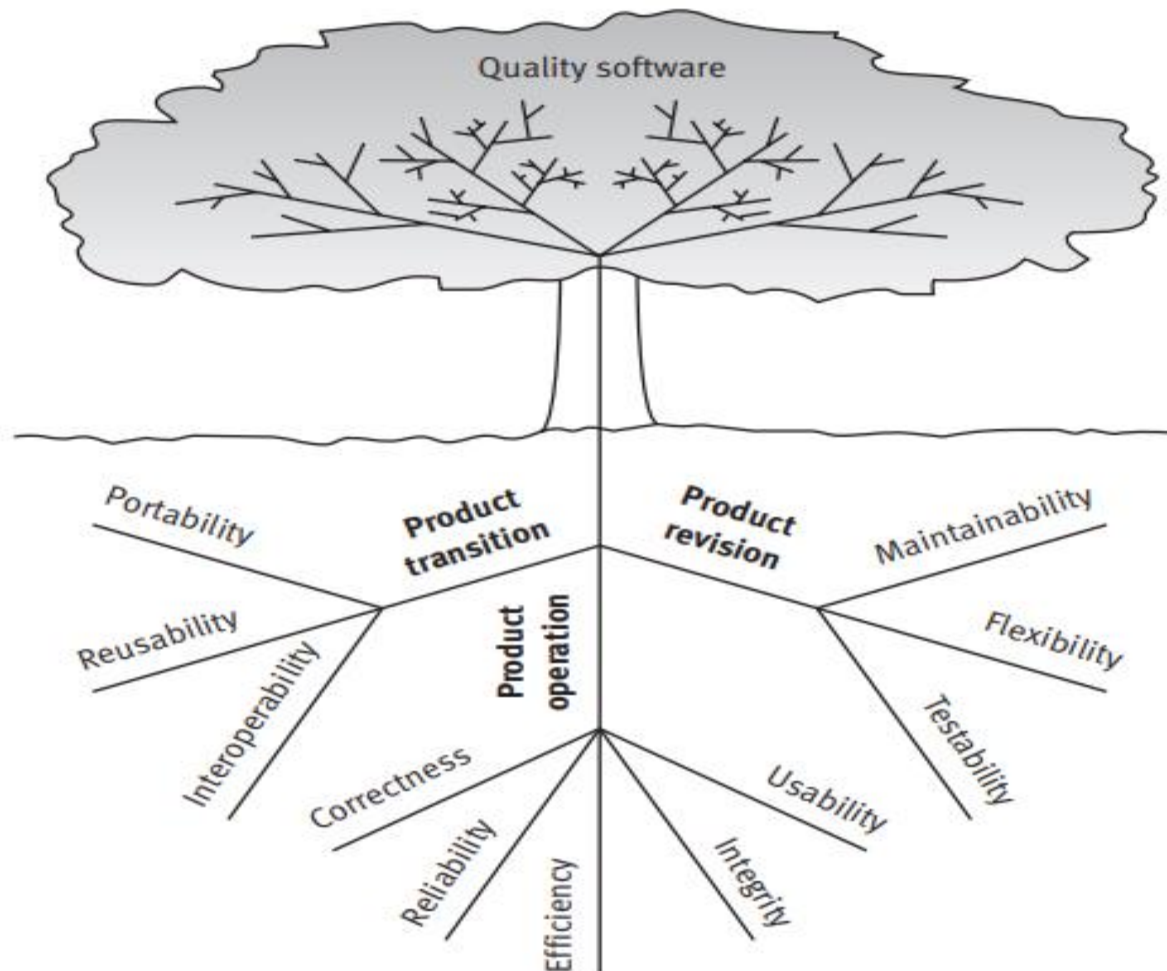
Why do we need them?

“Correctness is the baseline, but Quality is the differentiator”

A system can fulfill 100% of its functional requirements and still be a business failure if quality factors are ignored

McCall's Factor Model

McCall's Factor Model



McCall's Factor Model

McCall has 11 factors, grouped into three categories:

- **Product Operation Factors:**
 - How well it runs....
 - Correctness, reliability, efficiency, integrity, and usability
- **Product Revision Factors:**
 - How well it can be changed, tested, and redeployed
 - Maintainability; flexibility; testability
- **Product Transition Factors:**
 - How well it can be moved to different platforms and interface with other systems
 - Portability; Reusability; Interoperability

Product operation factors

1. Correctness

The extent to which a system satisfies its specifications and fulfills its mission through required outputs

Dimensions of Correctness:

- **Output Mission:** Delivering the specific required results (e.g., invoices, alarms)
- **Accuracy & Completeness:** Ensuring outputs are precise and based on whole data
- **Up-to-dateness:** Minimizing the time lag between an event and its system record
- **Availability:** Fast reaction time to requests or automated triggers
- **Standards Compliance:** Adhering to specific coding and documentation protocols

Product operation factors

2. Reliability

Reliability requirements deal with failures to provide service

Example specs:

- A heart monitoring system must have a failure rate of less than one per million cases
- Downtime for a system will not be more than ten minutes per month

Product operation factors

3. Efficiency

Efficiency requirements deal with the hardware resources needed to perform all the functions of the software system

Example of hardware resources:

- MIPS, MHz (cycles per second)
- Data storage capabilities measured in MB or TB
- Communication lines (usually measured in KBPS, MBPS, or GBPS)
- Time between recharging of portable units

Product operation factors

4. Integrity

Integrity requirements deal with the software system security

Huge nowadays; Cyber Security; Internet security; network security, and more. These are certainly not the same!

5. Usability

Usability requirements deal with the scope of staff resources needed to train a new employee and to operate the software system

Product revision factors

These factors deal with requirements that affect software maintenance activities:

- Maintainability
- Flexibility
- Testability

Product revision factors

1. Maintainability

- **Effort for Resolution:** Determines the time and effort needed by users and maintenance staff to identify software failure causes, correct them, and verify the successful fix
- **Structural & Documentation Requirements:** Focuses on the software's modular structure, internal program documentation, and the quality of the programmer's manual
- Example specs: size of module ≤ 30 statements

Product revision factors

2. Flexibility

- **Adaptive Capabilities:** The effort and resources (e.g., man-days) required to tailor a software package for different customers, scales of activity, or product ranges
- **Perfective Evolution:** Support for continuous improvement of services and adaptation to changes in the technical or commercial environment

Product revision factors

3. Testability

- **Facilitating the Testing Process:** Includes special features like log files and predefined intermediate results that assist testers in verifying system accuracy
- **Automated & Maintenance Diagnostics:** Involves automatic checks performed before system startup to report component faults and diagnostic tools used by technicians to identify failure causes
- Example specs: Standard Test Data

Product transition factors

Three quality factors pertain to the adaptation of software to other environments and its interaction with other software systems:

- Portability
- Reusability
- Interoperability

Product transition factors

1. Portability

- **Environmental Adaptation:** Requirements focus on adapting software to different hardware, operating systems, and diverse environments
- **Continued and Simultaneous Use:** Portability ensures the same basic software can be used across diverse situations or run simultaneously on different platforms
- Example specs: Cross-Platform Transfer

Product transition factors

2. Reusability

- **Cross-Project Application:** Focuses on using software modules originally designed for one project in new or future software development projects
- **Strategic Advantages:** Aims to save development resources and shorten production periods by utilizing existing, higher-quality modules where faults have already been detected and corrected
- Example specs: Future-Proofing Modules

Product transition factors

3. Interoperability

- **System Integration:** Focuses on creating functional interfaces between the software and other external systems or equipment firmware
- **Standards & Compatibility:** Specifications often define industry-standard output structures or the specific names of third-party software/firmware required for communication
- Example specs: Standardized Data Output, API, REST

Alternative models for quality factors

Alternative models for quality factors

Developed in the late 1980s as alternatives to the classic McCall's Model (1977)

Evans and Marciniak Model (1987):

- Consists of 12 factors
- Classified into 3 categories

Deutsch and Willis Model (1988):

- Consists of 15 factors
- Classified into 4 categories

Key Comparison to McCall:

- Both models exclude the Testability factor.
- Together, they introduced 5 new factors to broaden the definition of quality

The 5 New Quality Factors

Verifiability (Both models):

- To define design and programming features that enable the efficient verification of the software's integrity
- **Example:** Modularity, simplicity, and adherence to documentation/programming guidelines

Expandability (Both models):

- To support future efforts required to serve larger populations, improve existing services, or add new applications
- **Note:** included in McCall's **Flexibility** factor

The 5 New Quality Factors

Safety (Deutsch & Willis):

- To eliminate hazardous conditions for equipment operators caused by errors in process control software
- **Example:** A chemical plant system controlling acid flow based on pressure/temperature; safety specs define reactions to danger and specific alarm types needed

The 5 New Quality Factors

Manageability (Deutsch & Willis):

- To provide administrative tools that support software modification throughout the development and maintenance lifecycles
- **Core Tools Examples:** Includes configuration management and formal software change procedures

The 5 New Quality Factors

Survivability (Deutsch & Willis):

- Survivability requirements refer to the continuity of service
- **Note:** similar to reliability factor in McCall's model

McCall vs. Alternative models

Content analysis shows the differences are smaller than they first appear:

- Expandability $\approx$ Flexibility
- Survivability $\approx$ Reliability
- Testability is often treated as a sub-element of Maintainability

The Truly "New" Contributions:

- **Verifiability** (Added by both models)
- **Safety** and **Manageability** (Unique to Deutsch & Willis)

The core difference is the **structural grouping** of factors, not the factors themselves

McCall vs. Alternative models

No.	Software quality factor	McCall's classic model	Alternative factor models	
			Evans and Marciniak	Deutsch and Willis
1	Correctness	+	+	+
2	Reliability	+	+	+
3	Efficiency	+	+	+
4	Integrity	+	+	+
5	Usability	+	+	+
6	Maintainability	+	+	+
7	Flexibility	+	+	+
8	Testability	+		
9	Portability	+	+	+
10	Reusability	+	+	+
11	Interoperability	+	+	+
12	Verifiability		+	+
13	Expandability		+	+
14	Safety			+
15	Manageability			+
16	Survivability			+

Summary

- Software quality factor
- McCall's Factor Model
- Alternative models for quality factors